

Questões

Observação: faça tratamento de erro em todas as questões. A exceção de arquivo não encontrado se aplica a todas.

1. (University of Helsinki)¹ O arquivo `numbers.txt` contém números inteiros, um número por linha. O conteúdo pode ser semelhante a este:

1	2
2	45
3	108
4	3
5	-10
6	1100
7	...etc...

Por favor, escreva uma função chamada `largest`, que lê o arquivo e retorna o maior número presente nele.

Observe que a função não recebe nenhum argumento. O arquivo com o qual você está trabalhando sempre será chamado `numbers.txt`.

2. (University of Harvard – adaptada)² Uma maneira de medir a complexidade de um programa é contar o número de linhas de código (LOC), excluindo linhas em branco e comentários. Por exemplo, um programa como:

1	<code># Dizer olá</code>
2	
3	<code>name = input("Qual é o seu nome? ")</code>
4	<code>print(f"Olá, {name}")</code>

tem apenas duas linhas de código, não quatro, já que a primeira linha é um comentário e a segunda linha é em branco (ou seja, contém apenas espaços). Isso não é muita coisa, então as chances são de que o programa não seja muito complexo. Claro, só porque um programa (ou mesmo uma

¹ Tradução livre

² Tradução livre

função) tem mais linhas de código do que outro, não significa necessariamente que seja mais complexo. Por exemplo, uma função como:

```
1 def is_even(n):  
2     if n % 2 == 0:  
3         return True  
4     else:  
5         return False
```

não é necessariamente duas vezes mais complexa que uma função como:

```
1 def is_even(n):  
2     return n % 2 == 0
```

mesmo que a primeira tenha (mais que) o dobro de linhas de código. Na verdade, a primeira pode ser considerada mais simples se for mais fácil de ler! Então, o número de linhas de código deve ser considerado com cautela.

Mesmo assim, em um arquivo chamado `lines.py` que leia um arquivo e exiba o número de linhas de código nesse arquivo, excluindo comentários e linhas em branco. Faça tratamento de erros caso o arquivo não exista.

Assuma que qualquer linha que comece com `#`, opcionalmente precedida por espaços em branco, é um comentário. (Um docstring não deve ser considerado um comentário.) Assuma que qualquer linha que contenha apenas espaços em branco é uma linha em branco.

3. (University of Helsinki)³ Escreva um programa que peça ao usuário para digitar um texto. Seu programa deve então realizar uma verificação ortográfica e imprimir um feedback para o usuário, de modo que todas as palavras incorretas sejam destacadas com asteriscos ao redor delas. Veja os dois exemplos abaixo:

```
1 Write text: We use ptython to make a spell checker  
2 We use *ptython* to make a spell checker
```

³ Tradução livre

1	Write text: This is acually a good and usefull program
2	This is *acually* good and *usefull* program

A diferença entre letras maiúsculas e minúsculas não deve influenciar o funcionamento do seu programa.

O modelo de exercício inclui o arquivo wordlist.txt, que contém todas as palavras que o verificador ortográfico deve aceitar como corretas.

4.(University of Helsinki)⁴ Escreva uma função chamada `store_personal_data(person: tuple)`, que recebe uma tupla contendo algumas informações de identificação como argumento.

A tupla contém os seguintes elementos:

- Nome (string)
- Idade (inteiro)
- Altura (float)

Esses dados devem ser processados e escritos no arquivo `people.csv`. O arquivo pode já conter alguns dados; a nova entrada deve ser adicionada ao final do arquivo. Os dados devem ser escritos no formato:

1	nome;idade;altura
---	-------------------

Cada entrada deve estar em uma linha separada. Se chamarmos a função com o argumento ("Paul Paulson", 37, 175.5), a função deve escrever a seguinte linha ao final do arquivo:

1	Paul Paulson;37;175.5
---	-----------------------

⁴ Tradução livre

5. (University of Helsinki)⁵ O arquivo `solutions.csv` contém algumas soluções para problemas de matemática:

1	Arto;2+5;7
2	Pekka;3-2;1
3	Erkki;9+3;11
4	Arto;8-3;4
5	Pekka;5+5;10
6	...etc...

Como você pode ver acima, em cada linha o formato é `nome_do_aluno;problema;resultado`. Todas as operações são de adição ou subtração e cada uma tem exatamente dois operandos.

Por favor, escreva uma função chamada `filter_solutions()` que:

1. Leia o conteúdo do arquivo `solutions.csv`.
2. Escreva as linhas que têm o resultado correto no arquivo `correct.csv`.
3. Escreva as linhas que têm o resultado incorreto no arquivo `incorrect.csv`.

Usando o exemplo acima, o arquivo `correct.csv` conteria as seguintes linhas:

1	Arto;2+5;7
2	Pekka;3-2;1
3	Pekka;5+5;10

As outras duas linhas estariam no arquivo `incorrect.csv`.

Escreva as linhas na mesma ordem em que aparecem no arquivo original. Não altere o arquivo original.

Referências

University of Helsinki. Python Programming MOOC 2023. Disponível em: <https://programming-23.mooc.fi/>.

⁵ Tradução livre

University of Harvard. Malan, David J. CS50's Introduction to Programming with Python.
Disponível em: <https://cs50.harvard.edu/python/2022/>